1. Objective

The goal of the **SimpleSmartLoader** is to upgrade the traditional *SimpleLoader* into a **lazy (on-demand) loader**, emulating OS-level demand paging behavior.
Unlike the earlier loader that eagerly mapped all PT_LOAD segments upfront, this loader defers loading until the program actually accesses memory — i.e., until a **SIGSEGV** occurs.

---

2. Implementation Overview

2.1 Architecture

The implementation consists of the following major components:

Component	Description
ELF Header Parser	Reads ELF32 headers and identifies all PT_LOAD segments.
Signal Handler (SIGSEGV)	Handles segmentation faults, which act as *page faults*. Allocates and maps memory lazily.
Page Management System	Tracks all mapped pages, prevents double mapping, and maintains page-level statistics.
Statistics Engine	Computes total page faults, allocations, and internal fragmentation (in KB).
Cleanup System	Unmaps, closes, and releases resources safely after execution.

---

3. Lazy Loading Flow

3.1 Normal Execution Flow

1. The loader reads the ELF executable (e.g., fib or sum).
2. Instead of preloading, it sets up a **SIGSEGV handler** using sigaction().
3. It directly jumps to the _start() function using:
4. int (*entry_fn)(void) = (int (*)(void))ehdr.e_entry;
5. entry_fn();
6. Accessing unmapped memory triggers a **segmentation fault**.

7. The loader intercepts it, allocates the required memory **page**, loads data, and resumes execution.

---

4. Page Fault Handling (SIGSEGV Handler)

4.1 Trigger Condition

Whenever the process accesses a page not yet allocated (inside a valid PT_LOAD segment), the OS sends a SIGSEGV.

4.2 Steps Performed

1. **Identify faulting address**:
2. void *fault_addr = si->si_addr;
3. uintptr_t page_start = (uintptr_t)fault_addr & ~(PAGE_SIZE - 1);
4. **Locate the corresponding ELF segment** via virtual address bounds.
5. **Allocate one physical page** lazily using:
6. mmap(page_start, PAGE_SIZE, PROT_READ | PROT_WRITE | PROT_EXEC,
7. MAP_PRIVATE | MAP_ANONYMOUS | MAP_FIXED, -1, 0);
8. **Load segment data** from ELF file for overlapping bytes.
9. **Zero-fill remaining page region** for partial data (like .bss).
10. **Adjust protection flags** (R/W/X) based on ELF header flags using mprotect().
11. **Update accounting**:
    - Increment page_faults
    - Increment page_allocs
    - Record page in mapped_pages[] for tracking.

---

5. Internal Fragmentation Calculation

After execution, fragmentation is computed as:

$$\text{Internal Fragmentation} = \sum_{i=1}^{n} \text{(Allocated bytes} - \text{Segment memsz)}$$

Each segment may occupy extra space because memory is allocated in page-sized units (4KB).
The loader reports:

- **Page Faults**
- **Page Allocations**
- **Internal Fragmentation (KB)**

---

6. Key Implementation Highlights

Feature	Description
Demand Paging Simulation	Achieved via SIGSEGV handler and per-page allocation using mmap.
Page Tracking	Dynamic array to store each mapped page and its corresponding segment index.
Robust Error Handling	All system calls (open, pread, mmap, etc.) have strict error checks.
Clean Resource Management	cleanup_and_exit() unmaps pages, closes file descriptors, and resets global state.
Dynamic Memory Structures	Realloc-based array resizing for scalability.
Accurate File Overlap Logic	Handles segments not aligned to page boundaries correctly (fixes common bugs in naive implementations).

---

7. Test Programs Used

- **fib.c** — Tests recursive function calls and instruction page loading.
- **sum.c** — Tests data segment allocation, array operations, and zero-fill behavior.

---

8. Output Example

result: 1024

Page faults: 3

Page allocations: 3

Internal fragmentation: 4.00KB (4096 bytes)

---

9. Error and Protection Handling

- **Valid Access** → Page fault → Page mapped → Execution resumes.
- **Invalid Access (outside PT_LOAD)** → Immediate termination.
- **Protection Fault (double-mapped page)** → Program safely exits.

---

10. Design Decisions

1. **Page granularity loading** instead of segment-level.
2. **SIGSEGV as page-fault trigger** instead of custom traps.
3. **Separate data structures** for segments[] and mapped_pages[].
4. **SA_NODEFER flag** used in sigaction to allow reentrant handling.

---

11. Summary

This implementation successfully reproduces a **miniature demand paging system** in user space.
It achieves lazy loading, accurate tracking of allocations and fragmentation, and clean termination — aligning with OS-level virtual memory behavior as described in Linux.
